SENIOR PROJECT II · MODULE 1 · PROJECT RESET

Project Reset Report

CEA Event Space Reservation System. A shared, verified picture of where the Lovable.dev prototype stands today, what it owes, what could hurt us, how healthy the backlog is, and what we build first.

TEAM

Backend Builders · Jonathan Abeje, Nathnael Bereketab, Giliad Dawite, Peter Collins

CLIENT

Howard University, College of Engineering and Architecture (CEA), Dean's Office

REPOSITORY

github.com/joniabeje/backend-builders · branch `main` at `f317c48`

BOARD

GitHub Project #5 “Backend Builders Project Board” · 15 items

REPORT DATE

October 7, 2026

METHOD

Full read of `src/`, configs and docs; GitHub API audit of issues and board; `npm` install, typecheck, lint and build run locally

CONTENTS

1. Executive summary and readiness verdict
2. Scope and method
3. Current system architecture
4. Technical debt, risks and unknowns
5. Backlog health assessment
6. Initial Senior Project II priorities
7. Appendix: verification log and traceability

01 Executive summary and readiness verdict

Verdict: a convincing UI prototype, not yet a product. Reset first, then build.

The repository is a Lovable-generated TanStack Start front end with five routes, two hard-coded data modules and a handful of inline rules. Everything runs in the visitor's browser. There is no API, no database, no login, no test, no CI and no server-side rule of any kind. Submitting a reservation flips a React flag; approving one edits an in-memory array that a page refresh discards.

The README, landing page and reservation form all describe capabilities (conflict detection, automatic notifications to IT, custodial and security, analytics, AWS deployment) that do not exist in the code. The team already knows this: `DEBT_AND_RISK.md` (September 17) and the backlog notes (September 23) are accurate and candid. This report confirms them against the code, adds what they missed, and turns them into a sequenced plan.

The most important finding is not technical. The team has produced excellent planning artifacts, but the board shows three tickets in progress or in review that have no branch or pull request behind them. Senior Project II starts with a well-groomed backlog and an unbuilt foundation.

Readiness by area

Front-end shell and routing	Ready
Requirements and backlog definition	Ready
Architecture decisions (backend, hosting, auth)	Undecided
Persistence and API	Absent
Authentication and authorization	Absent
Business rules (validation, risk, recommendation)	Client-only, duplicated
Tests and CI	Absent
Reproducible install and build	Lockfile out of sync
Documentation accuracy	Drifted, partly corrected
Delivery flow (PRs, reviews, board accuracy)	Stalled

5
routes, all public, all client-side

0
API endpoints, tables, tests, CI workflows

~1,125
lines of application code (routes, lib, nav, router)

4,360
lines of UI scaffold with zero imports

15 / 0
issues open / closed

0
pull requests, ever

Five things to do in the first two weeks

1. **Make the three platform decisions** (backend runtime, hosting, auth provider) in one meeting and record them as ADRs. Every P0 ticket is blocked on them. Recommendation in section 6.
2. **Build the verification harness and CI** (#15) before any feature work, and turn on branch protection so the board's own Definition of Done becomes enforceable.

3. **Fix reproducibility**: regenerate the lockfile, delete the second one, pick one package manager, and add a `.gitattributes`. Today `npm ci` fails on a clean checkout and lint fails on every Windows machine.
4. **Reset the board to the truth**: move #3, #6 and #7 back to Ready, fill Priority and Size on #13 to #15, add milestones, and remove assignees from items that cannot start yet.
5. **Start #13 (persistent API)** on day three. It is the top of the ordered backlog and unblocks eight of the fifteen tickets.

02 Scope and method

This report covers the state of the project on October 7, 2026. It was produced by reading every application source file, every configuration file and every document in the repository, by querying the GitHub repository, issues and Project board through the GitHub CLI, and by running the project's own toolchain locally. Claims in the README and issue bodies were checked against the code rather than taken as given.

INPUT	WHAT WAS EXAMINED	STATE AT TIME OF REVIEW
Repository `main`	23 commits; working tree at `f317c48` plus uncommitted edits to `README.md`, `docs/system-map.html` and `docs/system-map.pdf`	Last commit September 24, 2026 (docs).
Application code	`src/routes/*` (6 files), `src/lib/*` (3), `src/components/Site*.tsx` (2), `src/router.tsx`, `src/components/ui/*` (46)	Fully read.
Configuration	`package.json`, both lockfiles, `vite.config.ts`, `wrangler.jsonc`, `tsconfig.json`, `eslint.config.js`, `bunfig.toml`, `components.json`	Fully read.
Documents	`README.md`, `DEBT_AND_RISK.md`, `docs/system-map.html` and its PDF	Fully read; findings re-verified.
GitHub Project #5	18 fields, 15 items, status, priority, size, assignees, board README	Queried via API.
GitHub issues	All 15 issues: bodies, labels, assignees, comments, milestones; all 29 labels; PRs; branches; branch protection	Queried via API.
Toolchain	`npm ci`, `npm install`, `tsc --noEmit`, `npm run lint`, `npm run build`	Run locally; results in Appendix A.

Out of scope: the Lovable-hosted preview environment, the CEA stakeholders' current manual process, and any code that exists only in a teammate's local checkout.

03 Current system architecture

The application is a single front-end bundle. TanStack Start renders a server-side HTML shell and then hydrates in the browser, but the project contains no server code of its own: no server functions, no API routes, no environment bindings. Every route imports its data from two static TypeScript modules compiled into the bundle, applies rules inline during render, and keeps any edits in component state.

3.1 Architecture diagram: as built

exists in the repository described in the README, not implemented

trust boundary (everything inside runs in the visitor's browser)

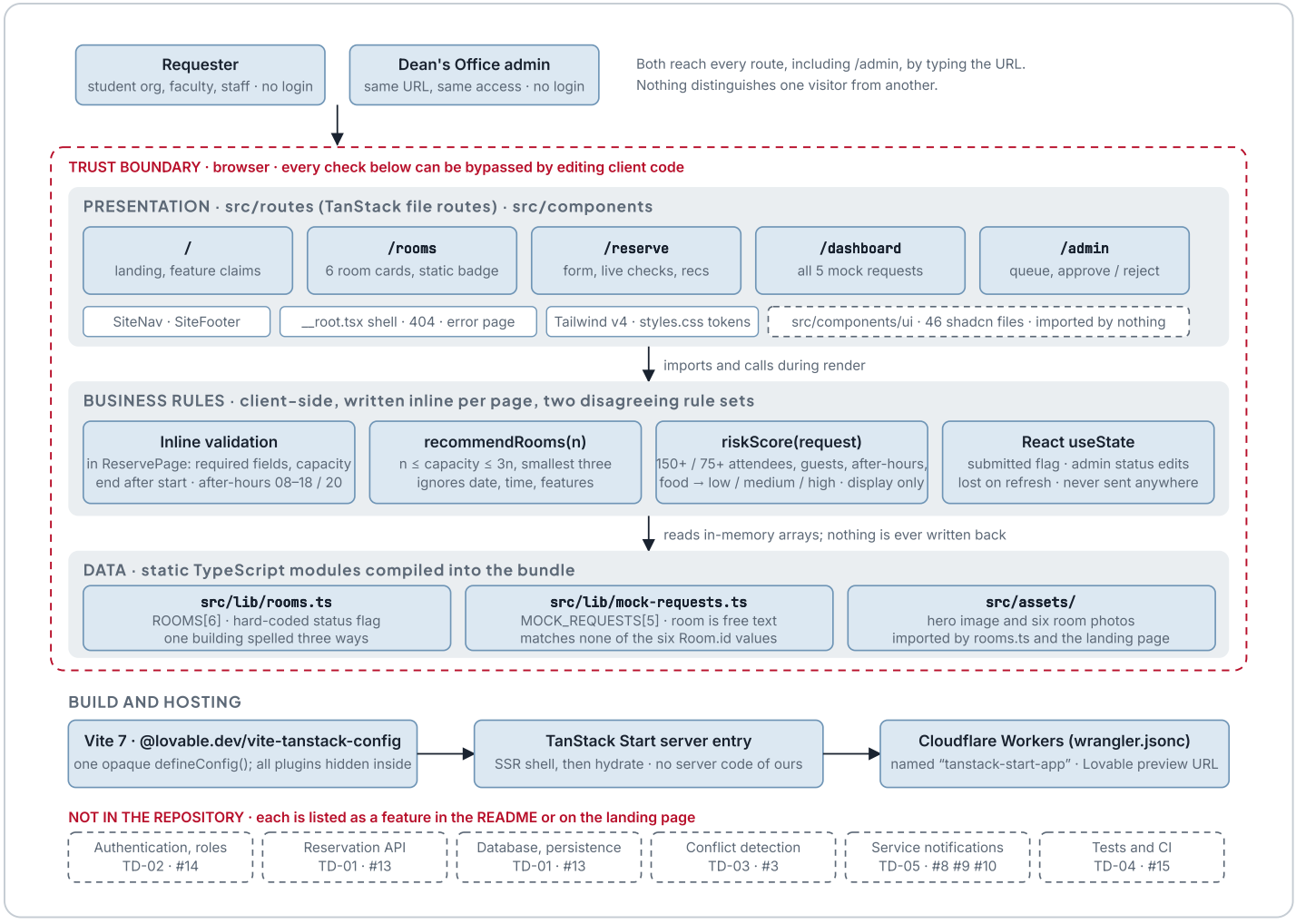


Figure 1. The system as it exists at commit `f317c48`. Three client-side layers inside one trust boundary, a build pipeline that serves the bundle, and six promised components that have no code. TD numbers refer to section 4; issue numbers to the GitHub backlog.

3.2 Component inventory

COMPONENT	LOCATION	LINES	WHAT IT DOES TODAY	NOTES
Root shell	`src/routes/__root.tsx`	75	HTML document, meta tags, Google Fonts, 404 page	Meta still says `twitter:site @Lovable`.

COMPONENT	LOCATION	LINES	WHAT IT DOES TODAY	NOTES
Router and error page	`src/router.tsx`	67	Creates the router from the generated route tree; default error component	Standard Lovable template code.
Landing page	`src/routes/index.tsx`	~240	Hero, feature grid, stakeholder sections, calls to action	Claims “10+ CEA Spaces”; the catalog has 6. Links straight to /admin.
Rooms page	`src/routes/rooms.tsx`	~115	Card grid of the six rooms with a static Available / Reserved badge	Badge never changes.
Reservation form	`src/routes/reserve.tsx`	~240	Twelve-field form, live validation sidebar, recommended-room buttons, success panel	Submit sets a boolean. No payload, no request ID, no network call.
Requester dashboard	`src/routes/dashboard.tsx`	~100	Status counters and one card per request	Shows all five mock requests to every visitor.
Admin dashboard	`src/routes/admin.tsx`	~160	Stats, status filter, request table with risk badges and approve / reject buttons	Edits `useState` only; no feedback capture, no audit.
Navigation, footer	`src/components/Site*.tsx`	60	Header with Home, Rooms, My Requests, Admin; footer	Admin link visible to everyone.
Room catalog and recommender	`src/lib/rooms.ts`	~35	`ROOMS[6]`; `recommendRooms(attendance)`	Capacity-only heuristic. Building spelled three ways.
Mock requests and risk score	`src/lib/mock-requests.ts`	~120	`MOCK_REQUESTS[5]`; `riskScore(request)`	Room names match none of the six room IDs. `afterHours` is stored, not derived.
Utilities	`src/lib/utils.ts`	6	`cn()` class-name helper	
UI scaffold	`src/components/ui/*`	4,360	46 shadcn/ui wrappers over Radix, Recharts, Embla, Vaul, cmdk and others	**Zero imports from any route or component.** Pure generator output.

3.3 Data flow: life of a request today

1. A visitor opens `/reserve`. The route component builds `FormState` in `useState` and recomputes `recommendRooms(attendance)` and an `issues[]` array on every keystroke.
2. Validation errors disable the submit button. Warnings (after-hours, external guests with 50 or more attendees, food) are shown but nothing records them.
3. On submit, the handler calls `setSubmitted(true)`. No object is built, nothing is posted, no ID is generated. The success panel promises “status updates as it moves through approval”.
4. The “View My Requests” link opens `/dashboard`, which renders the same five hard-coded `MOCK_REQUESTS` for every visitor. The request just “submitted” is not among them.
5. On `/admin`, approve and reject call `setStatus()`, which maps over a `useState` copy of `MOCK_REQUESTS`. A refresh restores the original five rows. No feedback, actor or timestamp is captured.
6. No notification, email, webhook or log is produced at any step, although the landing page, the form's tip box and the README all say otherwise.

3.4 Dependencies

Runtime and framework

React 19.2, React DOM	used
TanStack Router 1.168, TanStack Start 1.167, router plugin	used
Tailwind CSS 4.2, tw-animate-css, clsx, tailwind-merge, class-variance-authority	used
lucide-react icons	used
TanStack React Query 5.83	installed, unused
zod 3.24, react-hook-form 7.71, @hookform/resolvers	installed, unused
25 Radix UI packages, recharts, embla-carousel, vaul, cmdk, input-otp, react-day-picker, react-resizable-panels, sonner, date-fns	only via unused scaffold

Build, platform and tooling

Vite 7.3, @vitejs/plugin-react, vite-tsconfig-paths	used
@lovable.dev/vite-tanstack-config 1.4+	single point of coupling
@cloudflare/vite-plugin, wrangler.jsonc	build-only; deploy target implied
TypeScript 5.8 (strict), ESLint 9, typescript-eslint, Prettier 3	used; unused-variable rules switched off
Package managers	both bun.lockb and package-lock.json committed
Hosting	Lovable preview URL; Cloudflare Workers config; README says AWS
External services (auth, DB, email)	none configured

The Lovable config package wraps TanStack Start, React, Tailwind, path aliases, the Cloudflare plugin, a dev-only component tagger and HMR bridge plugins. The project's `vite.config.ts` is a single call with no arguments, so every build decision lives in a third-party package whose minor versions have already drifted from the committed lockfile (see TD-08).

3.5 Target architecture the backlog describes

For orientation, this is the architecture the fifteen tickets add up to. It is unchanged from the README and `docs/system-map.pdf`; the number on each component is the issue that delivers it.

exists today · planned, with the issue that adds it · quality gate

trust boundary moves to the server

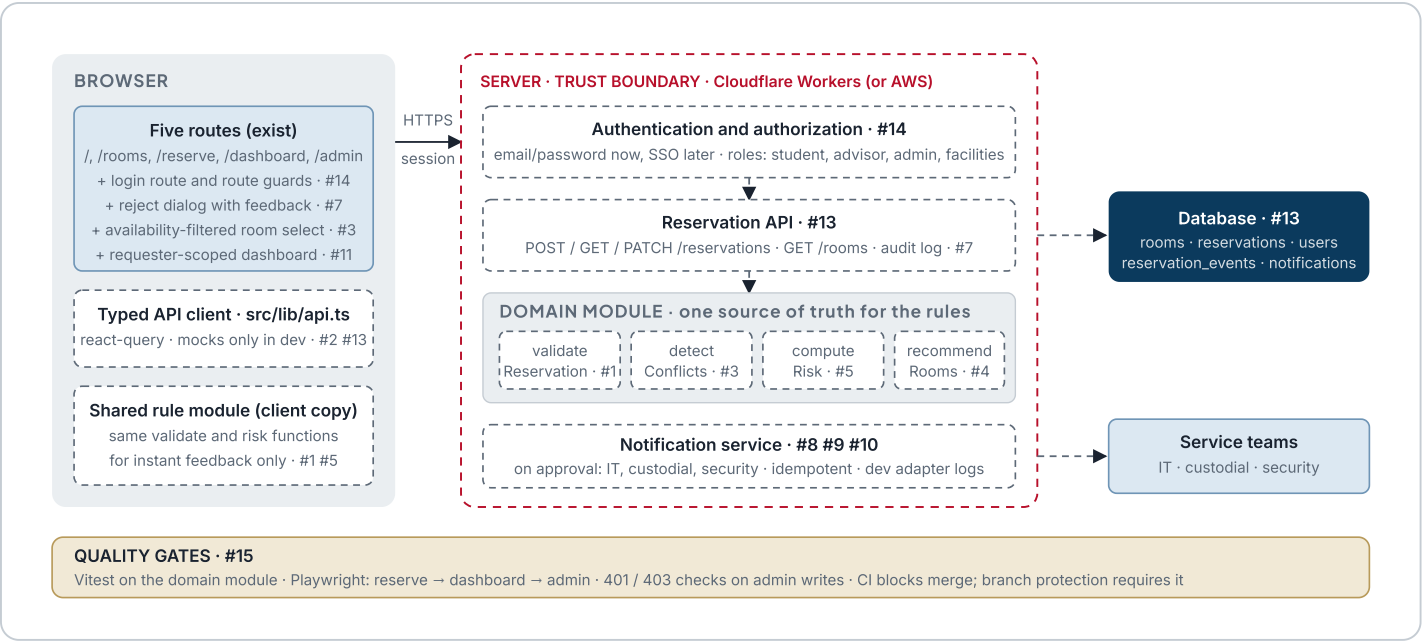


Figure 2. Where the backlog is heading. The rules move behind the server trust boundary and run once; the browser keeps a copy only for instant feedback. The full target diagram with ticket dependencies is in `docs/system-map.pdf`.

04 Technical debt, risks and unknowns

Items TD-01 to TD-06 and RISK-01 to RISK-03 come from `DEBT_AND_RISK.md` and were re-verified line by line for this report; every one still holds. Items TD-07 onward and R-04 onward are new findings from this review.

4.1 Technical debt register

ID	ITEM	CATEGORY	EVIDENCE (VERIFIED OCTOBER 7)	IMPACT IF LEFT AS IS	SEVERITY	RESOLVED BY
TD-01	Mock-only data plane; no API, no database	Architecture	`reserve.tsx` submit handler is `setSubmitted(true)`; `admin.tsx` mutates `useState(MOCK_REQUESTS)`; no DB or HTTP client in `package.json`.	No reservation is ever stored. Two users cannot see the same data, so scheduling, approvals and audit are impossible. Blocks 8 of 15 tickets.	Blocker	#13
TD-02	No authentication or authorization	Architecture, security	All five routes public; no `beforeLoad` guards; Admin link in nav for everyone; landing hero links to /admin.	Anyone with the URL can approve or reject events. Requester data (organization, advisor, guest flags) is visible to all. Cannot be deployed to a real audience.	Blocker	#14
TD-03	Business rules inline in components, two disagreeing rule sets	Architecture	Form after-hours: start ≥ 18:00 or < 08:00, end ≥ 20:00; guests + 50 attendees warns. `riskScore`: thresholds 150 / 75, reads a stored `afterHours` flag. Neither is enforced on submit.	Form and admin page can disagree about the same request. Rules can be bypassed from the browser. Every new page re-implements them and diverges further.	High	#1, #5
TD-04	Zero automated tests, no CI	Test	No `*.test.*` files, no test runner, no `.github/` directory, no branch protection on `main`.	Any refactor or AI-generated change can break scheduling or risk logic unnoticed. The board's Definition of Done ("CI green") cannot be met by any ticket.	Blocker	#15
TD-05	Documentation drift	Documentation	README “Key Features” lists conflict detection, notifications, analytics; landing page claims “10+ CEA Spaces” and automatic team notifications; form tip box repeats the claim. README architecture section was rewritten to the target design but not labelled as target.	Stakeholders expect features that do not exist; demos disappoint. AI coding agents reading the README generate calls to an API that is not there.	Medium	new issue (section 5.4)

ID	ITEM	CATEGORY	EVIDENCE (VERIFIED OCTOBER 7)	IMPACT IF LEFT AS IS	SEVERITY	RESOLVED BY
TD-06	Hard-coded inventory and broken joins	Data model	“L.K. Downing Hall”, “L.K. Downing Engineering Building”, “Downing Hall” for one building; `EventRequest.room` is free text (“L.K. Downing 1018”, “Mackey Atrium”) matching none of the six `Room.id` values; `Room.status` never changes.	Conflict detection has no key to join on, so double bookings cannot be detected. Availability badges mislead users.	Medium	#13, #3
TD-07	**New.** 46-file UI scaffold with zero consumers	Dead code, dependency weight	`src/components/ui/*` is 4,360 lines; a grep for `components/ui` outside that folder returns nothing. It pulls in 25 Radix packages plus recharts, embla, vaul, cmdk, input-otp, react-day-picker, react-resizable-panels and sonner.	Four times more code to read, lint and audit than the app itself. Roughly 30 dependencies to keep patched for no benefit. Confuses new contributors about what is in use.	Medium	new issue
TD-08	**New.** Non-reproducible install	Build	`npm ci` fails on a clean checkout: the lockfile pins `@lovable.dev/vite-tanstack-config 1.2.0` against a `^1.4.0` range, and is missing nitro, miniflare, h3, rolldown and 15 other packages. A binary `bun.lockb` is also committed (`bunfig.toml` disables the text lockfile), so neither lockfile can be reviewed in a diff.	CI cannot be set up until this is fixed (CI uses `npm ci`). Each developer resolves different versions, so "works on my machine" bugs are guaranteed.	High	#15 step 6
TD-09	**New.** Generator identity residue	Hygiene	Package name `tanstack_start_ts` ; Worker name `tanstack-start-app` ; `twitter:site @Lovable` meta; sample requesters are the team's own names; "<1s Validation" and "24/7 Tracking" marketing tiles.	Looks unfinished to a client or grader. Wrong Worker name collides if two teams deploy from the same template.	Low	new issue
TD-10	**New.** Quality gates switched off	Tooling	`@typescript-eslint/no-unused-vars` is `off` ; `noUnusedLocals` and `noUnusedParameters` are `false` . Dead code such as TD-07 cannot be caught by the linter.	Dead code and forgotten imports accumulate silently; review burden grows.	Low	#15
TD-11	**New.** Single opaque build config	Dependency coupling	`vite.config.ts` is `defineConfig()` from the Lovable package with no arguments. Adding a test runner, env bindings or a D1 binding means learning what that package injects first.	Every backend step in #13 starts with reverse-engineering a third-party config. A breaking release of that package halts all builds.	Medium	ADR-001, #13

ID	ITEM	CATEGORY	EVIDENCE (VERIFIED OCTOBER 7)	IMPACT IF LEFT AS IS	SEVERITY	RESOLVED BY
TD-12	**New.** Lint is not portable across operating systems	Tooling	No `.gitattributes`; Prettier enforces LF. On a Windows checkout `npm run lint` reports 5,546 line-ending errors and nothing else; on macOS or Linux it passes.	Windows teammates cannot run the gate the Definition of Done relies on, so they either skip lint or commit noisy whole-file reformatting.	Medium	#15 (add `* text=auto eol=lf` and renormalise)

4.2 Risk register

ID	RISK	LIKELIHOOD	IMPACT	EARLY SIGNAL	MITIGATION AND OWNER
RISK-01	Fabricated capability: docs and UI promise features the code lacks, so stakeholders and AI coding agents build on assumptions	High (already observed)	High	A demo question the UI cannot answer; an AI-generated PR that calls an API that does not exist	README “Current vs Planned” split; Definition of Done requires a test; no claim without an issue and a test. Owner: whoever edits README.
RISK-02	Open admin plus sensitive event data (organizations, advisors, guest flags); later, prompt injection if LLM features are added; fairness of risk heuristics	High	High	Any deployment of the current build to a real audience	#14 before any public URL; audit log (#7); thresholds agreed with CEA policy owner (#5); human confirms high-risk approvals.
RISK-03	Platform coupling: Lovable config package, TanStack Start 1.x, Cloudflare plugin, pre-release nitro dependencies, drifted lockfile	Medium-High	High	`npm ci` already fails; a Lovable config minor release changes build behaviour	Regenerate and pin the lockfile; own the Vite config (ADR-001); keep a non-Lovable deploy path exercised in CI.
R-04	**New.** Schedule: 8 P0 tickets (3 L, 4 M, 1 S) plus 7 more before the feature set is complete; one semester remains	High	High	Sprint 1 ends with no merged PR	Fix MVP scope now (section 6); weekly merged-PR count as the one tracked metric; cut P2 service integrations to a single shared notification module if behind.
R-05	**New.** Undecided platform: README says Node or Flask and AWS, repo says Cloudflare Workers, issue #13 proposes TanStack server functions + D1. Every P0 ticket waits on this	Medium	High	#13 still unassigned two weeks from now	One 30-minute decision meeting in week 1; record ADR-001 to ADR-003. Owner: Scrum Master.
R-06	**New.** Howard SSO may not be available to a student team	Medium	Medium	No response from Howard IT by end of October	Build #14 on email/password with a provider that supports SAML/OIDC later (Supabase Auth or Auth0); treat SSO as a P2 swap.

ID	RISK	LIKELIHOOD	IMPACT	EARLY SIGNAL	MITIGATION AND OWNER
R-07	**New.** Uneven contribution and bus factor: three members have commits, one has none; all board assignments sit with three people	Medium	Medium	Same author on every PR in a sprint	Pair on #13; rotate reviewer; every member owns at least one P0 ticket by sprint 2.
R-08	**New.** Lovable regeneration overwrites hand-written code if the team keeps editing through the Lovable editor while adding a backend	Medium	High	A commit by `gpt-engineer-app[bot]` after server code lands	Decide in week 1: Lovable for UI-only changes on a branch, or stop using it. Branch protection with required review blocks silent bot pushes to `main`.
R-09	**New.** Policy ambiguity: after-hours window, “large event” threshold and risk weighting are team guesses, not CEA rules; wrong guesses flag some student organizations unfairly	Medium	Medium	Two rule sets still disagree when #5 starts	Ask the Dean's Office for the written policy before #5; encode it once in `src/lib/risk.ts`; document in README.

4.3 Unknowns to resolve

UNKNOWN	WHY IT MATTERS	HOW TO FIND OUT	NEEDED BY
Backend runtime and database	Blocks #13 and therefore eight tickets	Team decision; recommendation in 6.2	Week 1
Hosting target (Cloudflare Workers vs AWS)	Determines DB options (D1 vs RDS), secrets handling, CI deploy step	Same meeting; check course requirements for AWS	Week 1
Auth provider and whether Howard SSO is obtainable	Shapes #14 and the user table	Email Howard IT / faculty advisor; pick a provider that can add SAML later	Week 2
CEA written policy: after-hours definition, attendance thresholds, who must approve what	Removes guesswork from #1, #5, #10; fairness	Interview the Dean's Office contact	Before #5
Real room inventory and current booking calendar	Seeds `rooms`; without it conflict detection has nothing to check	Request a spreadsheet or calendar export from facilities	Before #3
How IT, custodial and security want to be notified (email, ticket system, shared calendar)	Shapes the adapter in #8	Ask each office; a dev adapter that logs is fine until then	Before #8
Whether the team keeps using Lovable to edit this repository	Determines branch strategy and the risk in R-08	Team decision	Week 1
Is Peter Collins active on the project?	Capacity planning; R-07	Ask	Week 1

05 Backlog health assessment

GitHub Project #5 holds all fifteen issues from the repository. Twelve are feature tickets created on April 29, 2026 and labelled F1 to F16; three are technical-debt tickets created on September 23 from `DEBT_AND_RISK.md`. On September 23 every issue body was extended with priority, size, dependencies, a “where to start” section, acceptance criteria and a definition of done, each verified against the repository. The board README defines P0 to P2, an ordered backlog, an MVP scope and a pickup rule.

5.1 Board snapshot (October 7, 2026)

#	TITLE	BOARD STATUS	PRIORITY	SIZE	ASSIGNEES	DEPENDS ON	OBSERVATION
13	Replace mock data layer with persistent reservation API	Ready	unset	unset	none	none	Top of the ordered backlog, yet unassigned; P0 / L only in body text
14	Add authentication and role-based route protection	Ready	unset	unset	none	#13	Unassigned; fields unset
15	Establish Vitest + Playwright verification harness	Ready	unset	unset	none	none	No dependency, size S, unassigned for two weeks
2	Event Submission Form	Ready	P0	M	none	#13, #1	In Ready although its dependencies are not Done
1	Real Time Validation System	Ready	P0	M	joniabeje, NathnaelBereketab	none	Two assignees on one M ticket
3	Room Availability Checker	In progress	P0	M	giliaddawite	#13	No branch, no commit, no PR; depends on #13 which has not started
7	Approve Reject Workflow	In progress	P0	M	joniabeje	#13, #14	No branch, no commit, no PR
6	Admin Dashboard	In review	P0	L	joniabeje	#13, #14	No PR exists to review; the UI part was built in April by Lovable
11	Event Status Tracking	Backlog	P1	S	none	#13, #14, #7	Correct
5	Risk Detection System	Backlog	P1	M	giliaddawite	#13	Assigned while in Backlog
4	Room Recommendation Engine	Backlog	P1	M	giliaddawite	#3	Assigned while in Backlog
8	Service Integration: Notification System	Backlog	P2	M	NathnaelBereketab	#13, #7	Assigned while in Backlog

#	TITLE	BOARD STATUS	PRIORITY	SIZE	ASSIGNEES	DEPENDS ON	OBSERVATION
9	Custodial Notification System	Backlog	P2	S	NathnaelBereketab	#8	Assigned while in Backlog
10	Security Notification System	Backlog	P2	S	NathnaelBereketab	#8, #5	Assigned while in Backlog
12	Reservation History and Analytics	Backlog	P2	L	none	#13, #7	Correct

Totals: Backlog 7 · Ready 5 · In progress 2 · In review 1 · Done 0. Pull requests: 0. Milestones: 0. Issue comments: 0. Estimate, Start date and Target date fields: empty on all 15 items.

5.2 Scorecard

DIMENSION	RATING	WHAT WE FOUND
Scope coverage	Good	Every feature in the README and every top-3 debt item has a ticket. Gaps: labels F6 (Verify Advisor), F9 (Admin Recommendations), F14 and F15 (cleaning tasks) exist with no issue; no ticket for deployment or hosting; no ticket for documentation drift (TD-05); no ticket for the unused scaffold (TD-07) or lockfile (TD-08, folded into #15 step 6).
Ticket quality	Good	All 15 have priority, size, dependencies, “where to start” with file and line references, checkbox acceptance criteria and a definition of done. Several mark criteria already met. This is unusually strong for a student backlog.
Prioritization	Good	P0 to P2 defined, an ordered table of all 15 items, and an explicit MVP of 8 tickets. Debt items are placed first, which matches the code.
Dependency modelling	Good / Fair in use	Dependencies are written in every body and in the board README, but the board's own rule (“do not pull from Backlog until dependencies are Done”) is violated by #2 in Ready and #3, #6, #7 in progress or review while #13 and #14 have not started.
Status accuracy	Poor	Three items claim active work with no branch, commit or PR. The board describes intent, not reality. Standups and burndown based on it will mislead.
Throughput	Poor	Zero issues closed, zero PRs, zero comments since the board was created. All movement has been planning.
Ownership	Fair	9 of 15 assigned, but the four tickets that must start first (#13, #14, #15, #2) are the unassigned ones. Six Backlog items are pre-assigned, which conflicts with the pickup rule and with pairing. One team member has no assignment and no commit.
Field hygiene	Fair	#13 to #15 have no Priority or Size on the board even though the body says P0 and L / M / S. No milestones or iterations, so there is no way to say “sprint 2” on the board. Date and estimate fields are unused.
Process guardrails	Poor	Every definition of done says “CI green”, but there is no CI, no branch protection and no PR template. As written, no ticket can be Done.
Traceability	Good	Debt tickets link to `DEBT_AND_RISK.md` anchors; feature labels F1 to F16 map to the original feature list; `docs/system-map.pdf` contains a ticket dependency graph.

> **Overall: healthy definition, unhealthy flow.** The backlog is complete, well-written and correctly ordered. What it lacks is truth in status, enforceable gates, and any evidence of execution. The fix is procedural and takes an afternoon; the hard part is then sustaining weekly merges.

5.3 Backlog gaps: tickets to add

PROPOSED TICKET	SOURCE	PRIORITY	SIZE	DEPENDS ON
Record platform decisions as ADRs (backend, hosting, auth) in `docs/ARCHITECTURE.md`	R-05, TD-11	P0	XS	none
Fix reproducible install: one package manager, regenerated lockfile, `npm ci` in CI	TD-08	P0	XS	none (can be #15 step 6, but deserves its own checkbox)
Deploy pipeline: CI deploys `main` to a non-Lovable URL (Workers or AWS)	RISK-03	P0	S	ADRs, #15
README “Current vs Planned”; remove unbacked claims from landing page and form tip	TD-05, RISK-01	P1	XS	none
Remove or justify unused UI scaffold and its dependencies	TD-07	P1	S	#15 (so the build is proven green before and after)
Advisor verification decision (F6): email confirmation vs checkbox	#1 step 2	P1	S	#1, #14
Rename generator defaults (package, Worker, meta tags)	TD-09	P2	XS	none
Decide on F9, F14, F15 labels: create issues or delete labels	5.2 coverage	P2	XS	none

5.4 Board actions (one afternoon, Scrum Master)

1. Move #3, #6 and #7 back to **Ready**. Nothing is in progress until a branch exists.
2. Set Priority P0 and Size L / M / S on #13, #14 and #15 so the board's grouping matches the body text.
3. Create four milestones and attach every issue: **M1 Foundation** (#13, #14, #15, ADRs, lockfile), **M2 MVP** (#1, #2, #3, #6, #7, deploy), **M3 Decision support** (#4, #5, #11), **M4 Service integration** (#8, #9, #10, #12).
4. Clear assignees from Backlog items. Assign #13, #14, #15 now, one owner each, with a named pair on #13.
5. Move #2 to Backlog until #13 exposes a stub endpoint, or split it: “form-to-payload mapping and stubbed client” (Ready) and “wire to API” (Backlog).
6. Add the eight tickets in 5.3.
7. Turn on branch protection for `main`: require a PR, one review and the CI status check. Add a PR template that asks for the issue number and the test that proves the change.
8. Add a **Definition of Ready** to the board README: dependencies Done or stubbed, owner assigned, size set, milestone set.

06 Initial Senior Project II priorities

The ordering below follows the board's existing P0 to P2 scheme and dependency graph. It adds dates, a Sprint 0 for decisions and hygiene, and an explicit MVP demo target so the semester has a checkpoint that cannot be argued with.

6.1 Sequencing

WINDOW	GOAL	TICKETS	EXIT CRITERION
Sprint 0 Oct 7 to Oct 20	Decide, stabilise, make the board true	ADR-001 to 003 · lockfile fix · #15 harness and CI · branch protection · board actions 5.4 · README Current vs Planned	`npm ci && npm test && npm run build` passes in CI on a PR; `main` is protected; three ADRs merged; board has milestones and no false "in progress".
Sprint 1 Oct 21 to Nov 3	Persistence	#13 (schema, `GET /rooms`, `POST` and `GET /reservations`, typed client, seed six rooms) · #1 (extract `validateReservation`, add unit tests)	A reservation submitted on `/reserve` survives a refresh and appears on `/dashboard` and `/admin` for a second browser.
Sprint 2 Nov 4 to Nov 17	Identity and approvals	#14 (login, roles, server-side session check) · #7 (`PATCH` with feedback and audit row) · #13 remainder (OpenAPI, integration test)	Anonymous visitor to `/admin` is redirected; non-admin `PATCH` gets 403 in a test; approve and reject persist with actor and timestamp.
Sprint 3 Nov 18 to Dec 1	MVP complete	#2 (real submit, request ID, error handling) · #3 (`detectConflicts`, filtered room select, 409) · #6 (admin on API data, guarded route) · deploy pipeline	**MVP demo**: log in, reserve a free room, see it pending, log in as admin, approve with feedback, see it approved, on a non-Lovable URL. Playwright covers the path.
Sprint 4 Dec 2 to Dec 15	Decision support (P1)	#5 (one rule set, risk stored at submit) · #11 (requester-scoped dashboard, feedback visible) · #4 (availability- and feature-aware recommendations)	Form and admin page agree on risk; recommendations never include a booked room.
Sprint 5+ January 2027	Service integration and analytics (P2)	#8 shared notification module with dev adapter · #9 · #10 · #12	Approval of a qualifying request produces one notification per service, idempotently; analytics page reads real history.

Sizes from the board: Sprint 1 carries L + M, Sprint 2 M + M, Sprint 3 M + M + L + S. If Sprint 1 slips, protect the MVP by deferring #4 and the P2 notifications to a single shared module rather than three tickets.

6.2 Decisions to make in week 1, with a recommendation

DECISION	OPTIONS ON THE TABLE	RECOMMENDATION	WHY
ADR-001 Backend runtime	TanStack Start server functions · separate Express service · Flask service	**TanStack Start server functions** (`createServerFn`) in this repository	Already installed and already deploys through `wrangler.jsonc`. One language, one repo, one deploy. Zero new infrastructure for a team that has not merged a PR yet. zod is installed for shared schemas.

DECISION	OPTIONS ON THE TABLE	RECOMMENDATION	WHY
ADR-002 Database and hosting	Cloudflare Workers + D1 · AWS (RDS or DynamoDB) · Supabase Postgres	**Cloudflare Workers + D1** (SQLite); revisit AWS only if the course requires it	Matches ADR-001 and the existing config; free tier; local emulation through wrangler. If AWS is mandatory, use Supabase Postgres from Workers rather than re-platforming.
ADR-003 Authentication	Supabase Auth · Auth0 · Clerk · hand-rolled sessions · Howard SSO	**Email/password through a hosted provider that supports OIDC/SAML later**; Howard SSO as a P2 swap once IT answers	#14 cannot wait on a university process. A provider with a SAML connector keeps the SSO door open without blocking the MVP.
Lovable usage	Keep editing via Lovable · UI-only on a branch · stop	**Stop pushing from Lovable to `main`**; allow UI experiments on a `lovable/*` branch merged by PR	Protects server code from regeneration (R-08) and forces every change through CI.
Package manager	npm · bun	**npm**; delete `bun.lockb` and `bunfig.toml`	bun is not installed on at least one team machine; GitHub Actions and Cloudflare tooling default to npm; text lockfile is reviewable.

6.3 Working agreements for Senior Project II

- **One metric:** merged PRs per week, reviewed at every standup. Target two in Sprint 0, three or more afterwards.
- **Definition of Ready:** dependencies Done or stubbed; owner; size; milestone; acceptance criteria unchanged from the body.
- **Definition of Done:** PR says `Closes #n`; CI green; one reviewer who is not the author; acceptance checkboxes ticked; README updated if a claim changes.
- **No status without a branch.** "In progress" requires a branch named `<issue>-short-title`; "In review" requires an open PR.
- **Verify before trust:** AI-generated code (Lovable, Cursor, Copilot) is a draft until a test proves it. Risk, auth and persistence changes get a human walkthrough in review.
- **Ask the client early:** policy thresholds, room inventory and notification preferences are requested in Sprint 0, not when the ticket starts.

> **What success looks like on December 1, 2026.** A student logs in, reserves a free room for a real date, and sees the request pending. An admin logs in, sees the risk flags, approves with a note, and the student sees it approved after a refresh. All of it on a URL the team controls, with a Playwright test that proves it on every pull request. Everything after that is decision support on top of a real system.

07 Appendix

A. Verification log

Commands run on October 7, 2026 against a clean checkout of `main` at `f317c48` with Node 22.14 and npm. The lockfile was restored to the committed version afterwards.

CHECK	COMMAND	RESULT
Reproducible install	`npm ci`	Fails · `EUSAGE` : lockfile out of sync with `package.json` (20 invalid or missing packages, led by `@lovable.dev/vite-tanstack-config` , `rollup` , `nitro` , `miniflare`).
Install with resolution	`npm install`	Passes · 538 packages in 31 s, but it rewrites `package-lock.json` , which is why the lockfile keeps drifting.
Type check	`npx tsc --noEmit`	Passes · strict mode, no errors.
Lint	`npm run lint`	Fails · 5,546 errors and 7 warnings. All 5,546 errors are `prettier/prettier` “Delete `␍`”: the repo has no `.gitattributes` , so a Windows checkout with `core.autocrlf=true` gets CRLF endings that Prettier rejects. The 7 warnings are `react-refresh/only-export-components` , six of them in the unused scaffold. Lint therefore passes on macOS and Linux and fails on Windows (TD-12).
Production build	`npm run build`	Passes · client in 9.5 s (main chunk 326 kB, 104 kB gzipped), SSR server bundle 38.6 kB. The build regenerates `src/routeTree.gen.ts` , so it should be gitignored or committed consistently.
Tests	`npm test`	No script · no test runner, no test files.
UI scaffold usage	`grep -rn "components/ui" src | grep -v "^src/components/ui/"`	0 matches.
CI and protection	`ls .github · gh api …/branches/main/protection`	No `.github` directory · 404, no protection rules.

CHECK	COMMAND	RESULT
Pull requests and milestones	`gh pr list --state all · gh api …/milestones`	0 PRs · 0 milestones.

B. Repository and board facts

Repository created	April 29, 2026	Last push	September 24, 2026
Commits on `main`	23	By `gpt-engineer-app[bot]` (Lovable)	14
Human committers	Jonathan Abeje (6), Nathnael Bereketab (1), Giliad Dawite (1)	Branches	`main` only
Issues	15 open, 0 closed, 0 comments	Labels	29 (9 GitHub defaults, F1 to F16, 4 debt labels); F4, F6, F9, F14, F15 unused
Project board fields	18, including Priority (P0 to P2), Size (XS to XL), Estimate, Start date, Target date	Status columns	Backlog · Ready · In progress · In review · Done

C. Traceability: debt and risk to backlog

FINDING	EXISTING TICKET	PROPOSED TICKET (5.3)	MILESTONE
`TD-01, TD-06`	#13, #3		M1, M2
`TD-02, RISK-02`	#14, #7		M1, M2
`TD-03, R-09`	#1, #5		M2, M3
`TD-04, TD-10, TD-12`	#15		M1
`TD-05, RISK-01`		README Current vs Planned	M1
`TD-07`		Remove unused scaffold	M2
`TD-08, RISK-03`	#15 step 6	Reproducible install; deploy pipeline	M1
`TD-09`		Rename generator defaults	M4
`TD-11, R-05`		ADR-001 to ADR-003	M1
`R-04, R-07`		Working agreements (6.3), not a ticket	ongoing
`R-06`	#14	SSO swap as P2 follow-up	M4
`R-08`		Lovable usage decision; branch protection	M1

D. Related documents

- `DEBT_AND_RISK.md`: original debt and risk inventory, September 17, 2026.
- `docs/system-map.pdf` (from `docs/system-map.html`): target architecture, layered as-is view, life of a request, ticket dependency graph, September 24, 2026.
- GitHub Project #5 README: priority definitions, ordered backlog, MVP scope, pickup rule.
- This report: `docs/project-reset-report.pdf` (from `docs/project-reset-report.html`).

Project Reset Report · Backend Builders · CEA Event Space Reservation System · October 7, 2026 · Generated from `docs/project-reset-report.html`.